Automatización de las pruebas y pipelines

(Naty's)

Estudiantes:

Moises Pereira V-31.039.711

Ismael Torres V- 30.304.367

Elias Armas V- 30.868.382

Santiago Loyo V-30.675738

Edgmairys Apostol V- 31.493.172

Sección: IN3103

Barquisimeto, 2026

Índice

Contenido

Fundamentos de la Automatización de Pruebas....3

¿Qué es la Automatización de Pruebas?....3
Pruebas Manuales vs. Pruebas Automatizadas....4
¿Cuándo SÍ Automatizar y cuándo NO vale la pena? (El Retorno de Inversión / ROI)....4
La Pirámide de Pruebas (Arquitectura de la Automatización)....5
Beneficios Clave para "Galletas Natys"....5

Arquitectura y Diseño de Scripts de Prueba....6

¿Qué es un Script de Prueba?....6
Principios de un Buen Diseño de Scripts....6
Estructura Interna de un Script de Prueba....7
Patrones de Diseño de Pruebas (El Enfoque Profesional)....7
Aplicación Práctica en el Proyecto (Galletas Natys)....8

Pipelines ("La Tubería de Software")....8

Pipeline....8
Tipos de Pipelines según su Propósito....9
A. Pipeline de Integración Continua (CI Pipeline)....9
B. Pipeline de Entrega Continua / Despliegue Continuo (CD Pipeline)....9
C. Pipeline de Pruebas Automáticas y Rendimiento (Testing / Performance Pipeline)....10
D. Pipeline de Seguridad (DevSecOps Pipeline)....10
Estructura Interna de un Pipeline (Anatomía Técnica)....10
Aplicación en el Proyecto "Galletas Natys"....11

Herramientas de Ecosistema (CI/CD y Pruebas)....11

¿Qué es el Ecosistema de Herramientas?....11
Las Herramientas Clave del Ecosistema divididas por categorías:....11
A. Orquestadores de Pipelines / CI/CD (Los Directores de Orquesta)....11
B. Herramientas de Pruebas de Carga y Rendimiento (El Foco de su Exposición)....12
C. Herramientas de Pruebas Funcionales, Unitarias y de API....12
D. Entornos de Servidores y Control de Versiones (La Base Operativa)....13

¿Cómo encajan todas estas herramientas en "Galletas Natys"?.....13
Estrategias Avanzadas e Integración Continua (CI/CD).....13
Integración Continua (CI - Continuous Integration).....13
Entrega Continua y Despliegue Continuo (CD - Continuous Delivery / Deployment).....14
Triggers y Eventos (Los Disparadores del Pipeline).....14
Ramas, Estrategias y Herramientas del Ecosistema DevOps.....15
A. Gestión de Ramas (Git Branching Strategies).....15
B. Herramientas del Ecosistema (Las que debes mencionar).....15
Reducción de Tiempo de Lanzamiento (*Time-to-Market*).....16
¿Qué son las Estrategias Avanzadas en CI/CD?.....16
Estrategias Clave de Despliegue Avanzado (Deployment Strategies).....16
A. Despliegue Blue/Green (Azul/Verde).....16
B. Despliegue Canario (Canary Releases).....17
C. Despliegue Rolling (Despliegue Progresivo).....17
Prácticas Avanzadas de Integración Continua (CI).....17
A. Análisis Estático de Código (Static Code Analysis).....17
B. Pruebas de Carga Shift-Left (Pruebas tempranas de rendimiento).....18
C. Control de Calidad por Puertas lógicas (*Quality Gates*).....18
Aplicación Práctica en el Proyecto "Galletas Natys".....18

Fundamentos de la Automatización de Pruebas

¿Qué es la Automatización de Pruebas?

- **Concepto:** Es el uso de software especializado (scripts, herramientas o frameworks) para ejecutar pruebas en una aplicación de manera automática, sin intervención humana constante. Por lo tanto es una técnica en el desarrollo de software que utiliza programas, scripts y herramientas especializadas para ejecutar casos de prueba de forma automática, comparando los resultados reales con los resultados esperados

- **El objetivo principal:** No es eliminar al 100% las pruebas manuales, sino **liberar tiempo de tareas repetitivas y tediosas**, permitiendo que el equipo de QA (Quality Assurance) se concentre en pruebas exploratorias complejas y en la lógica de negocio.

Pruebas Manuales vs. Pruebas Automatizadas

Para tener clarísima la comparativa debemos saber las siguientes ventajas o desvenaas y sus funcionamientos dentro de las fases o estapas:

- **Pruebas Manuales:**
    - ***Cómo funcionan:*** Un humano (tester) se sienta frente a la computadora, abre la app de *Galletas Natys*, hace clic en los botones, llena formularios a mano y revisa si el resultado es el esperado.
    - ***Desventajas:*** Son lentas, costosas a largo plazo, propensas al error humano (el tester se cansa y pasa por alto un detalle) y imposibles de repetir idénticamente cientos de veces en segundos.
- **Pruebas Automatizadas:**
    - ***Cómo funcionan****:* Se escriben scripts o rutinas de código que simulan al usuario o prueban el sistema por debajo (código, APIs, rendimiento).
    - ***Ventajas****:* Son extremadamente rápidas, se pueden ejecutar mil veces exactamente igual, se pueden programar para correr de madrugada y detectan errores de forma inmediata.

¿Cuándo SÍ Automatizar y cuándo NO vale la pena? (El Retorno de Inversión / ROI)

Esta es una pregunta clásica que se plantean todo QA, DebOps de ingeniería de software. No todo se debe automatizar:

- **SÍ se debe automatizar cuando:**
    - Son tareas **repetitivas** que se ejecutan una y otra vez en cada versión (ej. el formulario de inicio de sesión o el cálculo del carrito de compras de Natys).

    - Son pruebas que requieren **altos volúmenes de datos** o simular muchos usuarios a la vez (aquí es donde entra JMeter).
    - Procesos críticos donde un error humano costaría mucho dinero.
- **NO se debe automatizar cuando:**
    - El software o la interfaz está cambiando constantemente (si cambias los botones de la web todos los días, el script automatizado se rompe y gastas más tiempo arreglándolo).
    - Pruebas de usabilidad o diseño visual subjetivo (¿se ve bonito este color? Eso lo juzga un humano).
    - Pruebas de "una sola vez" (si vas a probar una función que no se volverá a usar, gastas más tiempo programando el script que haciendo la prueba a mano).

La Pirámide de Pruebas (Arquitectura de la Automatización)

En el desarrollo profesional, las pruebas automatizadas se estructuran en una pirámide de base ancha y punta angosta:

1. **Base (Pruebas Unitarias - PHPUnit):** Son la base de la pirámide. Son muchas, rápidas y baratas de ejecutar. Prueban funciones aisladas del backend (ej. verificar que la función que descuenta stock en la BD de Natys funcione).
2. **Centro (Pruebas de Integración y API - Postman / Endpoints):** Comuevan que los módulos conversen bien entre sí (ej. que la interfaz web se comunique correctamente con la base de datos de MySQL).
3. **Cúspide (Pruebas de UI / Interfaz Gráfica - Selenium / Cypress):** Son más lentas y costosas de mantener. Simulan al usuario final haciendo clic en la pantalla de la tienda.
4. *(Y cruzando todo esto de manera transversal están las **Pruebas de Carga y Rendimiento con JMeter**, que evalúan cómo se comporta toda la estructura bajo presión).*

Beneficios Clave para "Galletas Natys"

- **Calidad garantizada:** Aseguramos que los pedidos, el inventario y los pagos no fallen ante la llegada de nuevos clientes.
- **Detección temprana de Bugs:** Encontrar un error en la fase de desarrollo es 10 veces más barato que encontrarlo cuando el sistema ya está en producción o en manos del cliente.
- **Escalabilidad:** Permite que el software crezca de manera sólida sin temor a que una nueva actualización rompa lo que ya funcionaba bien.

Arquitectura y Diseño de Scripts de Prueba

¿Qué es un Script de Prueba?

- **Concepto a fondo:** Un script de prueba es un conjunto de instrucciones escritas en un lenguaje de programación (como PHP, JavaScript o archivos de configuración) que le indica a una herramienta automatizada (como JMeter, PHPUnit o Selenium) exactamente **qué acciones realizar, qué datos enviar y qué resultados esperar** de la aplicación.
- **En el contexto de Galletas Natys:** No es más que el guion automatizado que le dice al sistema: *"Entra a la URL de login, ingresa el usuario 'admin', pon la contraseña '1234', haz clic en 'Enviar' y verifica que el sistema devuelva el panel de control con código HTTP 200"*.

Principios de un Buen Diseño de Scripts

Para que un script sea profesional y de alta calidad, debe cumplir con ciertos pilares de ingeniería:

- **Modularidad:** Dividir las pruebas en pequeñas piezas reutilizables. Por ejemplo, en vez de escribir el código para iniciar sesión en 50 scripts diferentes, creas un **módulo o función única de inicio de sesión** que todos los demás scripts puedan llamar cuando lo necesiten.
- **Independencia:** Los scripts de prueba **no deben depender unos de otros**. La prueba del carrito de compras debe poder ejecutarse sola, sin necesidad de que

obligatoriamente se haya ejecutado la prueba del catálogo primero (o si lo requiere, el script debe preparar sus propios datos de prueba de forma automatizada).

- **Mantenibilidad:** Si cambias el diseño de un botón o el nombre de un campo en la base de datos de *Galletas Natys*, deberías cambiar el selector o la variable en un solo lugar del script, no en cincuenta archivos distintos.
- **Robustez y Manejo de Errores:** El script debe saber qué hacer si la página tarda más de lo normal en cargar (implementando tiempos de espera o *timeouts*) para evitar falsos positivos (que una prueba falle solo porque el servidor estaba un segundo lento).

Estructura Interna de un Script de Prueba

Independientemente de la herramienta (ya sea una prueba unitaria en PHPUnit, una petición en Postman o un Thread Group en JMeter), todo script bien diseñado sigue una estructura de **fases**:

1. **Fase de Preparación (*Setup / Pre-conditions*):**
    - Se inicializan las variables, se configuran las conexiones (a la base de datos MySQL o al servidor web) y se limpian estados anteriores para asegurar un entorno limpio.
2. **Fase de Ejecución (*Action / Execution*):**
    - Se realiza la acción principal que se quiere probar (ej. enviar una petición HTTP POST al módulo de pedidos de Galletas Natys con 5 cajas de galletas de chocolate).
3. **Fase de Verificación (*Assertions / Checkpoints*):**
    - Es el corazón de la prueba. Aquí el script compara el resultado obtenido contra el resultado esperado. (Ej: *¿El servidor respondió con un código 200 OK? ¿El stock en la base de datos se redujo en 5?*). Si no coincide, la prueba falla.
4. **Fase de Limpieza (*Teardown / Post-conditions*):**
    - Se cierran conexiones abiertas, se eliminan los datos de prueba basura que se crearon en la base de datos durante la ejecución para dejar el sistema tal como estaba antes.

Patrones de Diseño de Pruebas (El Enfoque Profesional)

En proyectos de desarrollo avanzados (como los frameworks modernos), los scripts se organizan bajo patrones específicos:

- **Patrón Page Object Model (POM):** Muy usado en pruebas de interfaz. Consiste en separar la lógica de la página web (los selectores de los botones, campos de texto) del código de la prueba en sí. Si la pantalla de "Registro de Clientes" cambia de diseño, solo actualizas el archivo del objeto de la página y los scripts de prueba siguen intactos.
- **Pruebas Orientadas a Datos (*Data-Driven Testing*):** En lugar de escribir un script rígido para un solo usuario, el script se diseña para leer una lista de datos desde un archivo (como un CSV o una base de datos) y ejecutar la misma prueba múltiples veces con diferentes escenarios (ej. probar el login de Natys con 20 correos distintos, unos válidos y otros inválidos, usando el mismo script).

Aplicación Práctica en el Proyecto (Galletas Natys)

1. **En JMeter (Pruebas de Carga):** El diseño del script de prueba se estructura mediante un *Thread Group* (usuarios virtuales), un *HTTP Request Sampler* (la petición al módulo de compras) y una *Response Assertion* (verificar que la página devuelva la confirmación de compra sin errores).
2. **En el Código (PHPUnit):** El diseño del script valida mediante aserciones lógicas que las funciones del modelo de inventarios calculen correctamente los subtotales.

Pipelines ("La Tubería de Software")

Pipeline

- **Concepto:** Un Pipeline (Tubería de CI/CD) es una estructura lógica y automatizada que define el conjunto de pasos, etapas (*stages*) y tareas (*jobs*) que deben ejecutarse secuencialmente o en paralelo desde que un desarrollador guarda un cambio en el código fuente hasta que dicho cambio se valida, empaqueta y despliega en el servidor.

Por lo tano en ingeniería de software y DevOps, un **Pipeline** (tubería) es una serie de pasos automatizados que ocurren de forma secuencial o paralela para llevar el código fuente desde la laptop del desarrollador (tu entorno local con PHP, MySQL y XAMPP) hasta el entorno de producción (el servidor web accesible por los clientes).

- **La analogía perfecta:** Imagina la línea de ensamblaje de una fábrica de galletas: la masa entra por un extremo, pasa por la máquina de moldear, luego por el horno, después por el control de calidad (pruebas) y finalmente sale empaquetada lista para vender. El Pipeline hace exactamente lo mismo, pero con **código de software**.
- **Por qué importa en Galletas Natys:** Evita que tengamos que subir archivos manualmente por FTP o correr comandos a mano en el servidor de la UPTAEB. Todo el flujo es estandarizado y seguro.

- **Enfoque DevOps:** Es el corazón operativo de la filosofía DevOps, ya que elimina el trabajo manual, reduce los errores de despliegue y estandariza los controles de calidad en cada versión del sistema.

Tipos de Pipelines según su Propósito

En el desarrollo moderno existen distintos tipos de pipelines especializados según la tarea que realizan:

A. Pipeline de Integración Continua (CI Pipeline)

- **Propósito:** Enfocado puramente en la **calidad y compilación del código**.
- **¿Qué hace?** Cada vez que un integrante del equipo sube un *commit* o abre un *Pull Request*:
    1. Descarga el código actualizado.
    2. Compila los archivos y verifica errores de sintaxis (PHP / JS).
    3. Ejecuta la suite de pruebas unitarias y de integración (PHPUnit, Postman).
    4. Realiza un análisis estático de código (calidad y seguridad).
- **Resultado:** Aprueba o rechaza el cambio antes de que se fusione a la rama principal (`main`).

B. Pipeline de Entrega Continua / Despliegue Continuo (CD Pipeline)

- **Propósito:** Enfocado en la **preparación, empaquetado y subida al servidor**.
- **¿Qué hace?** Toma el código ya validado por el CI Pipeline y:
    1. Genera los artefactos de producción (empaqueta el sistema, minifica archivos CSS/JS, prepara la imagen o contenedor).
    2. Aplica migraciones de base de datos MySQL (si las hay).
    3. Despliega los archivos automáticamente en el servidor web (XAMPP/Apache local o servidor en la nube).
- **Resultado:** La nueva versión del sistema queda disponible para los usuarios finales sin interrupción del servicio.

C. Pipeline de Pruebas Automáticas y Rendimiento (Testing / Performance Pipeline)

- **Propósito:** Ejecutar pruebas de carga, estrés o regresión pesadas sin frenar el flujo diario de trabajo.
- **¿Qué hace?** Se programa para ejecutarse en horarios específicos (ej. todas las madrugadas) o mediante disparadores manuales:
    1. Despliega la aplicación en un entorno de pruebas (*Staging*).
    2. Lanza los scripts de **JMeter** simulando peticiones simultáneas sobre el backend de *Galletas Natys*.
    3. Mide latencia, picos de consumo de CPU/RAM y tasa de errores.
    4. Genera reportes dinámicos (dashboards) y notifica los resultados.

D. Pipeline de Seguridad (DevSecOps Pipeline)

- **Propósito:** Detectar vulnerabilidades de seguridad en el código y en las dependencias.
- **¿Qué hace?** Analiza el código buscando agujeros de seguridad (como inyecciones SQL o XSS en los formularios de Natys) y verifica que las librerías utilizadas no tengan fallos conocidos.

Estructura Interna de un Pipeline (Anatomía Técnica)

Independientemente de la herramienta (GitHub Actions, Jenkins, GitLab CI), todo pipeline se organiza en tres niveles jerárquicos:

1. **Pipeline (Workflows):** El flujo completo definido en un archivo de configuración (usualmente escrito en YAML).
2. **Etapas / Fases (*Stages*):** Las agrupaciones lógicas del proceso.
    - *Stage 1: Build* (Compilación)
    - *Stage 2: Test* (Ejecución de pruebas)
    - *Stage 3: Deploy* (Despliegue)
3. **Tareas y Pasos (*Jobs & Steps*):** Comandos individuales que ejecuta el servidor (ej. `npm install`, `phpunit`, `docker build`).

Aplicación en el Proyecto "Galletas Natys"

Para nuestro Sistema hemos realizado lo siguiente:

"En nuestro proyecto Galletas Natys, unificamos la Integración Continua y el Rendimiento: cada actualización pasa por una etapa de validación de sintaxis y pruebas unitarias de backend en PHP, seguida por la fase de pruebas de API y la ejecución programada de scripts de estrés en Apache JMeter para garantizar estabilidad antes de llevar los cambios a producción."

Herramientas de Ecosistema (CI/CD y Pruebas)

¿Qué es el Ecosistema de Herramientas?

- **Concepto:** Es el conjunto interconectado de plataformas, servidores, frameworks y librerías que permiten automatizar el ciclo de vida del software. Ninguna herramienta lo hace todo sola; se conectan entre sí (por ejemplo, GitHub se conecta con un servidor de pruebas, y este a su vez con JMeter o bases de datos).

Las Herramientas Clave del Ecosistema divididas por categorías:

A. Orquestadores de Pipelines / CI/CD (Los Directores de Orquesta)

Estas herramientas son las que automatizan la "tubería" (el pipeline) y ejecutan los scripts de forma automática cuando subes código:

- **GitHub Actions:** (La que más aplica si alojan su código en GitHub). Es nativa, fácil de configurar mediante archivos YAML y ejecuta tareas en la nube cada vez que hay un `push` o `pull request`.
- **Jenkins:** El servidor de automatización de código abierto clásico y más robusto de la industria. Se instala en servidores locales o dedicados (como un LAMP en Ubuntu) y permite crear pipelines hiper-personalizados.
- **GitLab CI/CD:** Integrado directamente en la plataforma GitLab, excelente para control de versiones y pipelines nativos.

B. Herramientas de Pruebas de Carga y Rendimiento (El Foco de su Exposición)

- **Apache JMeter:** (La estrella de su exposición). Herramienta open-source en Java para realizar pruebas de carga, estrés, análisis de rendimiento y simulación de múltiples usuarios concurrentes sobre servidores web y APIs.
- **k6 (Grafana):** Una herramienta moderna basada en JavaScript para pruebas de carga orientada a desarrolladores.
- **Apache Bench (ab):** Una herramienta de línea de comandos más ligera y básica para medir peticiones por segundo en servidores HTTP.

C. Herramientas de Pruebas Funcionales, Unitarias y de API

- **PHPUnit:** El framework estándar para ejecutar pruebas unitarias automatizadas en códigos desarrollados en **PHP** (ideal para validar la lógica del backend de *Galletas Natys*).
- **Postman / Newman:** Postman permite diseñar y probar APIs (endpoints de bases de datos), y **Newman** es su versión de consola para automatizar esas mismas pruebas dentro de un pipeline de CI/CD.

- **Selenium / Cypress:** Herramientas para automatizar pruebas de interfaz de usuario (UI) en el navegador web.

D. Entornos de Servidores y Control de Versiones (La Base Operativa)

- **Git y GitHub:** Control de versiones para almacenar el código fuente y disparar los eventos del pipeline.
- **Stack LAMP (Linux, Apache, MySQL, PHP / XAMPP):** El entorno donde corre localmente o en servidor el sistema web de *Galletas Natys*.

¿Cómo encajan todas estas herramientas en "Galletas Natys"?

Para la exposición, puedes explicarlo con este ejemplo práctico que demuestra integración total:

1. **Desarrollo:** El equipo programa usando PHP, JavaScript y MySQL sobre un entorno local con Apache/XAMPP.
2. **Control:** El código se sube a un repositorio central en **GitHub**.
3. **Automatización (CI/CD):** Un pipeline configurado con **GitHub Actions** detecta el cambio, compila el código y ejecuta pruebas automáticas en PHP.
4. **Rendimiento:** Periódicamente o antes de entregar a producción, se utiliza **Apache JMeter** para simular tráfico masivo sobre el servidor de *Galletas Natys*, generando gráficos de estrés y reportes dinámicos.

Estrategias Avanzadas e Integración Continua (CI/CD).

Integración Continua (CI - Continuous Integration)

- **Concepto:** Es la práctica de desarrollo donde los miembros de un equipo (Moises, Elías, Edgmairys, Ismael, Santiago) fusionan (*merge*) sus cambios de código en un repositorio central (como GitHub) de manera frecuente, idealmente varias veces al día.

- **Cómo opera técnicamente:** Cada vez que alguien hace un `git push` (un *commit*), un servidor de integración continua detecta el cambio automáticamente y lanza una serie de procesos de validación:
    1. **Compilación / Construcción:** Verifica que la sintaxis de PHP y JavaScript esté correcta y no rompa nada.
    2. **Pruebas Automáticas:** Ejecuta de manera inmediata las suites de pruebas unitarias (ej. PHPUnit probando las matemáticas del carrito de compras) y pruebas de API/endpoints (ej. Postman validando las rutas de la base de datos MySQL).
- **Beneficio clave:** El "infierno de las integraciones" desaparece. Si alguien sube un código roto, la CI lo detecta en segundos antes de que dañe la versión principal del sistema.

Entrega Continua y Despliegue Continuo (CD - Continuous Delivery / Deployment)

- **Concepto:** Es la extensión natural de la CI. Una vez que el código pasa todas las pruebas automatizadas del pipeline, la fase de CD se encarga de preparar y empaquetar la aplicación.
- **Diferencia clave entre Delivery y Deployment:**
    - *Continuous Delivery (Entrega):* El código está siempre listo y empaquetado para ir a producción, pero un humano (el administrador) da el último clic para aprobar la salida en vivo.
    - *Continuous Deployment (Despliegue):* Es 100% automatizado. Si pasa las pruebas, el pipeline sube los cambios al servidor web de producción de forma automática, sin intervención humana.
- **Aplicación en Galletas Natys:** Al hacer cambios en el diseño o en la lógica de pedidos, el CD empaqueta los archivos limpios y actualiza el servidor en la nube donde corre el sistema de la tienda.

Triggers y Eventos (Los Disparadores del Pipeline)

- **Concepto a fondo:** Un *Trigger* (disparador) es el evento que da inicio a todo el ciclo de la tubería. Es la chispa que enciende el motor del pipeline.
- **Tipos comunes de Triggers:**
    - **Eventos de Git:** Un `git push` hacia la rama `main` o `develop`.
    - **Pull Requests (PR):** Cuando un compañero abre una solicitud para fusionar código, el pipeline se dispara automáticamente para evaluar si ese código nuevo es apto o trae errores (*checks* de GitHub Actions).
    - **Triggers Programados (Cron):** Ejecutar pruebas de estrés (como las de JMeter) automáticamente todas las madrugadas para vigilar la salud del servidor.

Ramas, Estrategias y Herramientas del Ecosistema DevOps

Para hablar con total propiedad técnica, debes dominar los siguientes términos de arquitectura y herramientas:

A. Gestión de Ramas (Git Branching Strategies)

- `main` (o `master`): La rama principal que contiene el código estable que ve el cliente final (producción).
- `develop`: La rama donde se integran las nuevas funciones que el equipo va programando.
- `feature/...`: Ramas temporales creadas por cada integrante para programar un módulo específico (ej. `feature/modulo-carrito-natys`).

B. Herramientas del Ecosistema (Las que debes mencionar)

- **GitHub Actions:** La herramienta nativa de GitHub para crear pipelines directamente en la nube mediante archivos de configuración en formato YAML (`.github/workflows/`). Es la más práctica para proyectos universitarios y profesionales.
- **Jenkins:** Un servidor de automatización open-source clásico de la industria, muy robusto, ideal para configuraciones locales y servidores dedicados.

- **GitOps:** Metodología donde la infraestructura y el despliegue del software se gestionan directamente a través de repositorios de Git.
- **DevOps:** La cultura y conjunto de prácticas que une el desarrollo de software (`Dev`) con las operaciones de sistemas (`Ops`) para automatizar todo el ciclo de vida.

Reducción de Tiempo de Lanzamiento (*Time-to-Market*)

- **Qué significa:** Es el indicador de negocio que mide cuánto tiempo transcurre desde que una idea o corrección de bug es programada por ti en tu laptop, hasta que el usuario final la está usando en la plataforma web o móvil.
- **El impacto de tu tema:** Sin pipelines ni automatización, este proceso tomaba días de pruebas manuales y revisiones tediosas. Con una tubería automatizada de CI/CD, el **Time-to-Market se reduce a minutos**, permitiendo que *Galletas Natys* despliegue actualizaciones, ofertas o parches de seguridad de manera inmediata y sin errores humanos.

¿Qué son las Estrategias Avanzadas en CI/CD?

- **Concepto:** Son metodologías, patrones de arquitectura y configuraciones de nivel superior que permiten que el proceso de Integración y Despliegue Continuo sea más seguro, tolerante a fallos, rápido y adaptado a sistemas en producción que no pueden darse el lujo de apagarse.
- **Por qué importan:** En un entorno real (como una tienda virtual de alta demanda), un despliegue mal hecho puede tirar el servidor y hacer perder ventas. Las estrategias avanzadas evitan esto.

Estrategias Clave de Despliegue Avanzado (Deployment Strategies)

Cuando el pipeline termina de compilar y probar, ¿cómo pasa el código a los usuarios? Aquí entran las estrategias profesionales:

A. Despliegue Blue/Green (Azul/Verde)

- **Cómo funciona:** Se mantienen dos entornos idénticos en producción: el **Entorno Azul** (la versión actual funcionando con la que interactúan los clientes) y el **Entorno Verde** (la nueva versión con las mejoras).
- **El proceso:** El pipeline despliega y prueba todo en el entorno Verde de forma silenciosa. Cuando se comprueba que todo está perfecto, un balanceador de carga redirige el tráfico de los usuarios del Azul al Verde en un segundo.
- **Ventaja:** Si algo falla en el Verde, el tráfico regresa inmediatamente al Azul (cero tiempo de inactividad o *zero downtime*).

B. Despliegue Canario (Canary Releases)

- **Cómo funciona:** La actualización no se le muestra a todo el mundo de golpe.
- **El proceso:** El pipeline lanza la nueva versión solo a un pequeño porcentaje de usuarios (por ejemplo, al 5% o al 10% de los clientes de *Galletas Natys*). El sistema monitorea los errores y el rendimiento. Si ese 10% navega sin problemas durante unas horas, el pipeline libera el cambio para el 100% de la plataforma.

C. Despliegue Rolling (Despliegue Progresivo)

- **Cómo funciona:** El sistema actualiza los servidores o nodos uno por uno o en pequeños grupos. Mientras unos servidores se actualizan, los demás siguen atendiendo peticiones, evitando caídas totales del servicio.

Prácticas Avanzadas de Integración Continua (CI)

Para que la integración continua sea verdaderamente avanzada y robusta, se implementan las siguientes capas dentro de la tubería:

A. Análisis Estático de Código (Static Code Analysis)

- Antes de ejecutar cualquier prueba, herramientas automatizadas analizan el código fuente línea por línea buscando malas prácticas, código obsoleto, vulnerabilidades de seguridad o fallas de sintaxis sin necesidad de ejecutar la aplicación.

B. Pruebas de Carga Shift-Left (Pruebas tempranas de rendimiento)

- Tradicionalmente, las pruebas de estrés con **JMeter** se hacían al final de todo el proyecto. La estrategia avanzada (*Shift-Left*) propone mover las pruebas de rendimiento hacia las fases tempranas del pipeline, ejecutando pruebas de carga ligeras automatizadas por cada cambio importante para detectar cuellos de botella antes de llegar a producción.

C. Control de Calidad por Puertas lógicas (*Quality Gates*)

- Son reglas estrictas configuradas en el pipeline. Por ejemplo: *"Si la cobertura de pruebas unitarias baja del 80%", o "si JMeter detecta una tasa de error mayor al 2% en las peticiones"*, el pipeline bloquea automáticamente el despliegue y emite una alerta roja al equipo de desarrollo.

Aplicación Práctica en el Proyecto "Galletas Natys"

"En Galletas Natys, aplicamos una estrategia de integración continua donde cada cambio en el repositorio activa un pipeline que valida la sintaxis, ejecuta pruebas unitarias y de API, y nos permite integrar pruebas de rendimiento con JMeter. Esto asegura que cualquier mejora en el backend o en el módulo de pedidos llegue a producción de forma automatizada, controlada y libre de fallos masivos."